2026-10-18 20:51:01 You

🟠 Level 6 — Roles

This is where Ansible starts becoming **production-style**.

58. **Why Roles?**
59. **Role structure**
60. **tasks/**
61. **handlers/**
62. **templates/**
63. **files/**
64. **vars/**
65. **defaults/**
66. **meta/**
67. **Role dependencies**
68. **Using roles in playbooks**

ChatGPT gpt-6

Ansible Level 6 — Roles

Goal: Understand how to organize Ansible code into reusable, maintainable components the way teams use it in production.

Core idea: A role groups related tasks, variables, templates, files, handlers, and dependencies into one reusable structure.

For example, instead of writing all Nginx installation and configuration tasks in every playbook, create an `nginx` role and reuse it across multiple projects.

58. Why Roles?

Problem without roles: A large playbook contains installation, user creation, configuration, file copying, and service restart tasks all together. Reusing the same logic becomes difficult.

Solution: Put related automation into a role.

Without roles:

```
</> Plain text

deploy.yml
   ├──── Install Nginx
   ├──── Create directories
   ├──── Copy configuration
   ├──── Create users
   └──── Restart service
```

With roles:

```
</> Plain text

site.yml
   ├──── nginx role
   ├──── app role
   └──── monitoring role
```

Benefits

- Reuse automation across multiple playbooks.
- Keep projects organized.
- Separate application configuration from deployment logic.
- Make code easier to test, maintain, and share.
- Support team collaboration.

Production example: One `nginx` role installs and configures Nginx on development, staging, and production servers, using different variables for each environment.

59. Role Structure

A typical role has this directory structure:

```
</> Plain text

ansible-project/
 ├──── inventory.ini
 ├──── site.yml
 └──── roles/
     └──── nginx/
         ├──── tasks/
         │     └──── main.yml
         ├──── handlers/
         │     └──── main.yml
         ├──── templates/
```

```
│   └── nginx.conf.j2
├── files/
│   └── index.html
├── vars/
│   └── main.yml
├── defaults/
│   └── main.yml
└── meta/
    └── main.yml
```

Each directory has a specific responsibility. You don't have to create every directory; include only the ones your role needs.

Create a role using:

```bash
ansible-galaxy role init roles/nginx
```

This scaffolds a role structure with standard directories and files.

60. `tasks/`

Purpose: Contains the main tasks the role executes.

File: `roles/nginx/tasks/main.yml`

```yaml
- name: Install Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: true
  become: true

- name: Ensure Nginx is running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
  become: true
```

Remember:

- `main.yml` is the default entry point for role tasks.
- Tasks run in order from top to bottom.

- You can split tasks into other files and import or include them from `main.yml`.

Production example: Install packages, create directories, deploy configuration, and ensure services are running.

---

61. `handlers/`



Purpose: Contains tasks that run only when notified by another task.

File: `roles/nginx/handlers/main.yml`

```yaml
</> YAML

- name: Reload Nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
  become: true
```

Notify the handler from `roles/nginx/tasks/main.yml`:

```yaml
</> YAML

- name: Deploy Nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/conf.d/myapp.conf
  become: true
  notify: Reload Nginx
```

How it works:

1. Ansible deploys the configuration.
2. If the file changes, the task reports a change.
3. Ansible notifies the handler.
4. The handler reloads Nginx at the appropriate point in the play.

Remember: Handlers normally run at the end of the play's task section, and multiple notifications of the same handler generally result in one execution.

Production example: Reload Nginx only when its configuration changes instead of reloading it on every run.

---

62. `templates/`

Purpose: Stores dynamic configuration files written using Jinja2.

File: `roles/nginx/templates/nginx.conf.j2`

```
</> Nginx

server {
    listen {{ nginx_port }};
    server_name {{ server_name }};
}
```

The role task:

```
</> YAML

- name: Deploy Nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/conf.d/myapp.conf
  become: true
  notify: Reload Nginx
```

Role templates are automatically searched in the role's `templates/` directory.

Remember:

- `templates/` → Dynamic files with variables or template logic.
- `files/` → Static files copied without Jinja2 rendering.

Production example: Generate environment-specific application configuration using different port and domain variables.

---

63. `files/`

Purpose: Stores static files that need to be copied to remote servers.

File: `roles/nginx/files/index.html`

```
</> HTML

<h1>Welcome to My Application</h1>
```

Role task:

```yaml
- name: Deploy homepage
  ansible.builtin.copy:
    src: index.html
    dest: /var/www/html/index.html
    owner: www-data
    group: www-data
    mode: '0644'
  become: true
```

Ansible looks for `index.html` in the role's `files/` directory.

Production example: Copy static HTML pages, scripts, certificates, or other static assets.

Remember: Use `copy` with `files/` for static content and `template` with `templates/` for dynamic content.

---

64. `vars/`

Purpose: Stores role variables that generally should not be overridden by ordinary inventory variables or role defaults.

File: `roles/nginx/vars/main.yml`

```yaml
nginx_service_name: nginx
nginx_config_path: /etc/nginx
```

Use these variables in a task:

```yaml
- name: Check Nginx configuration directory
  ansible.builtin.stat:
    path: "{{ nginx_config_path }}"
  register: nginx_config_stat
```

Important:

- Variables defined in `vars/main.yml` have relatively high precedence.
- Inventory variables, play variables, and role defaults generally cannot override these role variables when their precedence is lower.

- Use `vars/` for values that should rarely be customized.

Production example: Store internal role-specific constants that should not normally vary between environments.

---

65. `defaults/`

Purpose: Stores default values that users can easily override.

File: `roles/nginx/defaults/main.yml`

```yaml
nginx_port: 80
nginx_server_name: localhost
nginx_package_name: nginx
```

Use them in `roles/nginx/templates/nginx.conf.j2`:

```nginx
server {
    listen {{ nginx_port }};
    server_name {{ nginx_server_name }};
}
```

Override defaults in a playbook:

```yaml
- name: Configure production web servers
  hosts: web
  vars:
    nginx_port: 8080
    nginx_server_name: app.example.com
  roles:
    - nginx
```

The role uses port `8080` and server name `app.example.com` instead of the defaults.

Remember:

defaults/	vars/
Lowest variable precedence among common role variable sources	Much higher precedence
Easy to override	Harder to override
Good for configurable settings	Good for values that should rarely change

Production example: Define a default application port, log path, or service setting while allowing different environments to customize it.

---

66. meta/

Purpose: Stores role metadata, including information about the role and its dependencies.

File: `roles/nginx/meta/main.yml`

```yaml
galaxy_info:
  role_name: nginx
  author: your_team
  description: Install and configure Nginx
  min_ansible_version: "2.14"

dependencies: []
```

Key uses:

- Describe the role.
- Declare supported information for Ansible Galaxy.
- Define dependencies on other roles.

Remember: `meta/main.yml` is not where you put ordinary task logic or application configuration values.

Production example: Document a reusable role before publishing it to an internal or public Ansible Galaxy repository.

---

67. Role Dependencies

Purpose: Automatically run prerequisite roles before the role that depends on them.

Example: Your application role requires Nginx to be installed and configured first.

Project structure:

```
</> Plain text

roles/
 ├──── nginx/
 │     └──── tasks/main.yml
 └──── myapp/
       ├──── tasks/main.yml
       └──── meta/main.yml
```

File: `roles/myapp/meta/main.yml`

```
</> YAML

dependencies:
  - role: nginx
```

File: `roles/myapp/tasks/main.yml`

```
</> YAML

- name: Create application directory
  ansible.builtin.file:
    path: /opt/myapp
    state: directory
    mode: '0755'
  become: true
```

Playbook:

```
</> YAML

- name: Deploy application
  hosts: web
  become: true
  roles:
    - myapp
```

Execution flow:

1. Ansible processes the `myapp` role dependency.
2. The `nginx` role runs first.
3. The `myapp` role runs after its dependency.

Production example: An application role depends on a common Linux configuration role, which prepares required users, packages, or directories.

Important: A role dependency is not the same as simply listing two roles in a playbook. Dependencies are declared in the role metadata and processed automatically. Ansible also has rules for dependency deduplication; execution can depend on parameters and dependency structure.

---

68. Using Roles in Playbooks

Purpose: Call one or more roles from a playbook instead of writing all the tasks directly in it.

Example project

```
</> Plain text

ansible-project/
├── inventory.ini
├── site.yml
└── roles/
    └── nginx/
        ├── tasks/main.yml
        ├── handlers/main.yml
        ├── templates/nginx.conf.j2
        └── defaults/main.yml
```

File: `site.yml`

```
</> YAML

- name: Configure web servers
  hosts: web
  become: true

  roles:
    - nginx
```

Inventory: `inventory.ini`

```
</> INI
```

```
[web]
web01 ansible_host=192.168.1.10
web02 ansible_host=192.168.1.11
```

Run:

```
</> Bash

ansible-playbook -i inventory.ini site.yml
```

Ansible executes the Nginx role against both hosts.

Passing variables to a role

```
</> YAML

- name: Configure web servers
  hosts: web
  become: true

  roles:
    - role: nginx
      nginx_port: 8080
      nginx_server_name: app.example.com
```

These values override the role's defaults.

Using multiple roles

```
</> YAML

- name: Deploy web application
  hosts: web
  become: true

  roles:
    - common
    - nginx
    - myapp
    - monitoring
```

Roles are applied in the listed order, subject to role dependencies and play execution rules.

Production example: A deployment playbook can use `common` to prepare the OS, `nginx` to configure the web server, `myapp` to deploy the application, and `monitoring` to install monitoring agents.

Quick revision — Level 6

Topic	What to remember
58. Why roles?	Reusable, organized automation
59. Role structure	Standard directories for different responsibilities
60. `tasks/`	Main task execution
61. `handlers/`	Run when notified
62. `templates/`	Dynamic configuration files
63. `files/`	Static files
64. `vars/`	Higher-precedence role variables
65. `defaults/`	Easily overridable default values
66. `meta/`	Metadata and role dependencies
67. Dependencies	Prerequisite roles
68. Playbook usage	Reuse roles across environments

Minimal hands-on practice

Build one `nginx` role with just these components:

- [ ] `defaults/main.yml` — Define the default port.
- [ ] `tasks/main.yml` — Install and start Nginx.
- [ ] `templates/nginx.conf.j2` — Generate a configuration.
- [ ] `handlers/main.yml` — Reload Nginx when the configuration changes.
- [ ] `site.yml` — Call the role and override the default port.

Run the playbook twice. The second run should normally report no changes if the server is already in the desired state.

Interview priority: Understand why roles are used, the difference between `defaults/` and `vars/`, `files/` versus `templates/`, handlers, role dependencies, and how a playbook invokes a role.